Final copy ✓CA

EE2-12

IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2012

ISE PART II: MEng, BEng. and ACGI

SOFTWARE ENGINEERING 2

Wednesday, 30 May, 2:00 pm

Time allowed: 1:30 hours

There are THREE questions on this paper.

Answer ALL questions.

Q1 carries 40% of the marks. Questions 2 and 3 carry equal marks (30% each).

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s): Madden,L.G.
Second Marker(s): Pitt,J.V.

@Imperial College London

The Questions

1.

All the parts of this question refer to Figure 1.1. The figure shows an architecture that is similar but is not identical to the analogue filter case study developed in EE2-12. Your answers should be based only on what is shown in the figure. For the question parts below use fully qualified names when you need to identify elements from the code extract.

a) Identify an example of each of the following:
   (i) A polymorphic variable.
   (ii) A polymorphic message.
   (iii) A template datatype.
   (iv) A pure virtual function.
   (v) An overridden class member.
   (vi) An overloaded class member.

[12]

b) Why is it counter-intuitive to implement the Orthodox Canonical Class Format (OCCF) members in the class `TFilter`?

[4]

c) Why is it helpful to implement OCCF in `TFilter`?

[4]

d) Assume that the `TFilter` stream operators have been implemented. Which Software Engineering principle can be used to justify *not* providing stream operators for the `TFilter` derived classes? Briefly explain your answer.

[4]

e) How many member function implementations are needed for the one class derived from `TFilter`: (excluding OCCF, getters and setters).
   (i) If inheritance was *not* used.
   (ii) If inheritance was used.

[4]

f) Excluding the OCCF members, identify the `TFilter` members that may be *invoked* directly using a Fully Qualified Name from within the code of the derived classes.

[4]

g) If the protocol of the `TFilter` class were extended to include a getter and a setter then which (if any) member function might be defined as a constant function and why?

[4]

h) If the `TFilter` destructor included a simple print statement then it is likely to reveal more runtime object instantiations than the number of explicit object instantiations shown in the code. Why?

[4]

```
class TFilter {              // extract from class definitions follows …
protected:              // House-keeping members
    double rVal, lVal, cVal;
    TFilter(void)                                                    // etc…
    TFilter(const TFilter &tf)                                       // etc…
    ~TFilter(void)                                                   // etc…
    TFilter & operator=(const TFilter &f)                            // etc…
    TFilter(double rVal, double lVal, double cVal)                   // etc…
public:                                          // Core member functions …
    double getF0(void)                                               // etc…
    virtual double getQ(void)                                        // etc…
    virtual complex <double> getGain(double) = 0;                    // etc…
}; // end TFilter

class TCircuit {
private:
    complex<double> comp1, comp2;                                    // etc…
}; // end TCircuit

class TBandPassFilter : public TFilter {
public:             // House-keeping members and Core member functions …
    TBandPassFilter(double rVal, double lVal, double cVal)           // etc…
    double getQ(void)                                                // etc…
    complex<double> getGain(double freq)                             // etc…
}; // end TBandPassFilter

int main(void) {            // extract from main program follows
      …
    typedef TFilter * TFilterType[];
    TFilterType filters = {new TBandPassFilter(1.0,2.0,1.0),
                           new TBandPassFilter(2.0,1.0,2.0)};
    complex <double> z1(0.0,0.0);
    for (int i = 0; i < 2; i++)
      z1=complex<double>(filters[i]>getGain(1.0)).real(),0.0);
                           //etc …
```

Figure 1.1 Code extract based on the EE2-12 Computing laboratories

2.

All the parts of this question refer to Figures 2.1 and 2.2. Figure 2.1 shows a problem domain description that is similar but is not identical to the nodal analysis case study developed in EE2-12. Your answers should be based only on what is shown in the figure. The figure shows a basic resistive circuit, the nodal analysis and a solution coded in Matlab™. Figure 2.2 shows one possible architecture to model the case study.

a) Write C++ *Specification* code that would be found in the *Specification files* for the *two* template classes shown in Figure 2.2. The code should include the relationship between the classes, the data members and the core member functions. Do not include OCCF or the getters and setters.

[8]

b) Choose *one* of the *two* template classes. For your chosen class write C++ *Implementation* code that would be found in an *Implementation file* for the *core* member functions. Do not include OCCF or the getters and setters.

[8]

c) Using the class you selected for your answer to (b) write the C++ *Implementation* code:

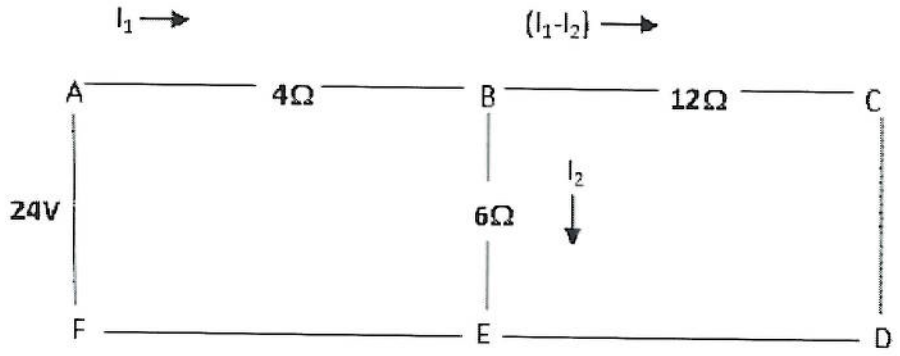
i) For the *Stream Insertion* operator function.

ii) For the *Stream Extraction* operator function.

[8]

d) Write a C++ *Exception class* to deal with an out-of-range index value provided as an actual argument to either `TMatrix::at()` *or* `TMatrix::put()`. Indicate whether your exception class is to be used with the getter or the setter.

[6]



Using Kirchoff's Laws for the loop ACDF: $16*I_1 - 12*I_2 = 24$ (Eqn. 1)

and for the loop ABEF: $4*I_1 + 6*I_2 = 24$ (Eqn. 2)

Solving (Eqn. 1) and (Eqn. 2) gives $I_1 = 3$ and $I_2 = 2$

The simulataneous equations may be solved in Matlab™ as follows

```
>> resistorsMatrix = [16 -12; 4 6];
   voltagesVector =  [24; 24];
   resistorsInverted = resistorsMatrix ^-1;
   currentsVector = resistorsInverted * voltagesVector
```

Note: ^-1 represents 2x2 matrix inversion.

Figure 2.1 Nodal analysis of a simple resistive circuit

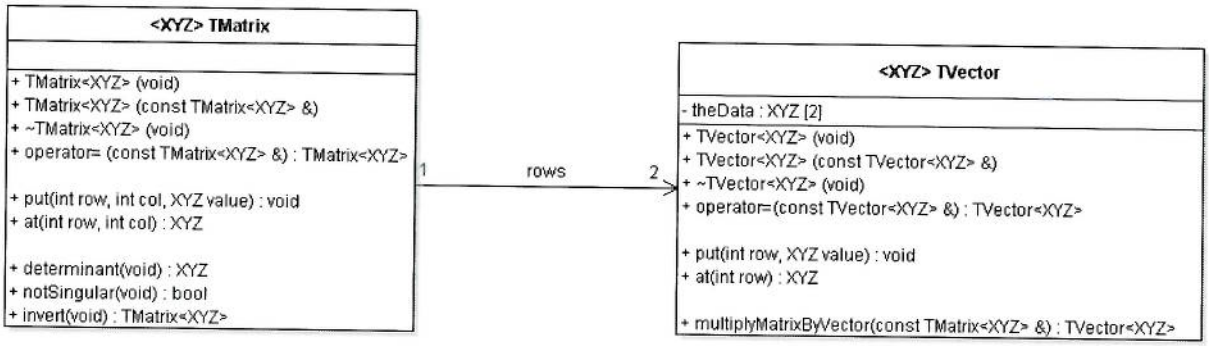


Figure 2.2 A proposed architecture for the case study

3.

All the parts of this question refer to Figure 3.1. The figure shows a problem domain description that is similar but is not identical to the robot world case study developed in EE2-12. Your answers should be based only on what is shown in the figure.

For all the UML class diagrams include the relationships between classes (where appropriate), the data members and *only* the core member functions. You do *not* need to include constructors, destructors, getters, setters, operators, helpers.

a) Based on a textual analysis of the problem description suggest names for an architecture that includes four candidate classes. Choose names that make it clear what each of the four classes are.

[4]

b) Draw a UML class diagram that could be passed on to a programmer for implementation in code.

Include as much detail as can be found in the problem description and no more. The following details are required:

- Class and association names.
- Association multiplicities.
- Indicate if the associations are uni-directional or bi-directional.
- Visibilities for every class member.
- Include enough class members to clarify the role of the class but you do not need to include every class member.

[12]

c) Two of the classes in your answer to (b) should have similar data members. The underlying data and the associated functionality can be factored out into a new class. Draw a partial class diagram that *only* shows the *new* class and the *two* classes identified in (b) and includes the same details as (b).

[10]

d) Do you consider the change of architecture described in (c) to be a valid development or not? Briefly justify your answer.

[4]

A robot moves around a two-dimensional grid. The position of a robot on the grid is specified using `X` and `Y` coordinates. A robot records its current position on the grid and is located at (0, 0) when instantiated. The robot protocol includes the following:

- set the `X` position;
- set the `Y` position;
- get the `X` position;
- get the `Y` position.

A robot may be free standing on the grid or it may be loaded onto a robot carrier for transportation to a grid location in order to perform some task. The robot carrier may carry up to four robots at a time. Like the robot, the robot carrier records its current position on the grid. Robots loaded onto the carrier have the same location as the robot carrier.

The robot carrier shares the same protocol as the robot but it can also include some additional messages:

- indicate how many places are available for carrying robots i.e. 0 to 4;
- add a robot to the carrier;
- remove a robot from the carrier;

Every robot has a fixed amount of data storage. The quantity of storage determines the type of the robot because a team leader robot has a large amount of storage and a drone robot has a small amount of storage. There must be exactly one team leader and between one and three drone robots on the robot carrier before it moves to a new location.

A drone robot can also respond to a number of additional messages:

- harvest the data (associated with the task to be performed by the drone);
- transfer the data to the team leader robot.

A team leader robot can also respond to an additional message:

- receive and store all the data harvested by the drone robots.

Figure 3.1 Problem domain description – Robot world

Master

EE2-12 - Solutions - 2012

Note: The answers below are much longer than required for pedagogic reasons. Most answers require no more than a sentence. The essence of this question is to briefly describe a concept with an example of the concept from the case study (often in the form of a FQN).

Q1.

(a) In the main program identify the following:

(i) A polymorphic variable.

`filters`

(ii) A polymorphic message.

`getGain(), getF0(), getQ()`

(iii) A template datatype.

`complex<T>`

(iv) A pure virtual function.

`getGain()`

(v) An overridden class member.

`getGain(), getQ()`

(vi) An overloaded class member.

- `operator=()` is overloaded for the base and any derived class since the second formal argument is always a different filter object. Therefore the function name is the same but the signatures are not the same.
- `TFilter` is overloaded for the constructors which each have a different set of formal arguments. Therefore the function name is the same but the signatures are not the same. [12]

(b) Why is it counter intuitive to implement OCCF in `TFilter`?

The class is a pure virtual (i.e. abstract) class therefore it can never be instantiated which would suggest that there is no need for instance related code e.g. constructors, assignment operators. [4]

(c) Why is it helpful to implement OCCF in `TFilter`?

Although the class can not be instantiated it can provide shared code to be invoked using the FQN from the code in any of the derived classes. [4]

(d) Which Software Engineering principle can be used to justify *not* providing any stream operators for the derived classes? Explain why this is the case.

According to Liskov's principle any derived class can substitute for a variable declared as an instance of the base class. The shared protocol, included the base class stream operators, may be invoked on the polymorphic variable. Justification based on Programming by Differences is also acceptable. [4]

(e) How many member function implementations are needed for the four classes derived from TFilter:

(i) If inheritance was *not* used.

```
TFilter::getF0(), TFilter::getQ(),TFilter::getGain(),
TBandPassFilter::getF0(),
TBandPassFilter::getQ(),TBandPassFilter:: getGain()
```

i.e. 6

(ii) If inheritance was used.

```
TFilter::getF0(),TFilter::getQ(),
TBandPassFilter::getQ(),TBandPassFilter::getGain()
```

i.e. 4

[4]

(f) Excluding the OCCF members, identify the `TFilter` members that can be *invoked* directly from the implementation code in the `TFilter` derived classes.

Looking for the explicit invocation of code (i.e. not just using inherited methods) e.g.

- the derived class constructor can invoke the base class constructor i.e. `TFilter(double,double,double)` using initialiser notation
- The only overridden member function is `TFilter:getQ()`.The derived class implementation of the MF can explicitly invoke the base class version of the MF using the FQN.

[4]

(g) If the protocol of the `TFilter` class were extended to include a getter and a setter then which (if any) member function might be a constant function and why?

Getters can be invoked on a constant object if the getter is a constant function and it does not attempt to change the state of the receiver. A setter could be declared as a constant function but it will generate a compilation error as soon as it attempts to change the state of the constant object.

[4]

(h) If the `TFilter` destructor included a simple print statement then it is likely to reveal more runtime object instantiations than the number of explicit object instantiations shown in the code. Why?

The *other* objects are implicitly created objects under the bonnet as a consequence of send-by-value in to and out of functions, anonymous temporary objects created for reasons of efficiency typically in object expressions and passing objects in to and out of functions etc...

[4]

Q2

a) [8]

```
template <typename XYZ> class TVector {
  public:
    const TVector<XYZ> multiplyMatrixByVector(const TMatrix <XYZ> &);

  private:
    XYZ theData[2];
};

template <typename XYZ> class TMatrix {
  public:
    XYZ determinant(void);
    bool notSingular(void);
    const TMatrix <XYZ> invert(void);

  private:
    TVector <XYZ> rows[2];
}
```

b) [8]

```
XYZ TMatrix::determinant(void){
    return (this->at(0,0)*this->at(1,1))-(this->at(0,1)*this->at(1,0));
}

bool TMatrix::notSingular(void){
    return (fabs(this->determinant()) > 0.0000001);
}

template <typename XYZ>
const TMatrix <XYZ> TMatrix::invert(void){
  if ((this->notSingular() == true)){
    TMatrix <XYZ> result;
    XYZ d = this->determinant();

    result.put(0, 0, this->at(1,1) / d);
    result.put(0, 1, -1*this->at(0,1)/d);
    result.put(1, 0, -1*this->at(1,0)/d);
    result.put(1, 1, this->at(0,0)/d);
    return result;
  }
}
== OR ==
template <typename XYZ>
const TVector<XYZ> TVector::multiplyMatrixByVector(const TMatrix <XYZ> &m){
    TVector <XYZ> result;
    XYZ top = this->at(0);
    XYZ bottom = this->at(1);
    for (int r = 0; r < 2; r++)
        result.put(r, (m.at(r, 0) * top) + (m.at(r, 1) * bottom));
    return result;
 }
====
```

c) i) For the *Stream Insertion* operator; [4]

```
template <typename XYZ> ostream & operator<<(ostream &o, const TMatrix <XYZ> &t){
    for (int r = 0; r < 2; r++)
        {
        for (int c = 0; c < 2; c++)
            o << setw(8) << setprecision(2) << t.at(r, c) << " ";
        o << endl;
    }
    return o;
}

== OR ==

template <typename XYZ> ostream & operator<<(ostream &o, const TVector <XYZ> &t){
    o << setw(6) << setprecision(3) << t.at(0) << endl << setw(6) << t.at(1) << endl;
    return o;
}
```

ii) For the *Stream Extraction* operator. [4]

```
template <typename XYZ> istream & operator>>(istream &i, TMatrix <XYZ> &tc)
{
    XYZ tmp;
    for (int r = 0; r < 2; r++)
        for (int c = 0; c < 2; c++)
        {
            i >> tmp; // might add prompt using row and col values
            tc.put(r, c, tmp);
        }
    return i;
}

== OR ==
template <typename XYZ> istream & operator>>(istream &i, TVector <XYZ> &tc)
{
    XYZ tmp1, tmp2;
    i >> tmp1 >> tmp2; // might add prompt using row and col values
    tc.put(0, tmp1);
    tc.put(1, tmp2);
    return i;
}
```

d) [6]

```
template <typename XYZ> class PutException : public runtime_error {
public:
    PutException(int row, int col, XYZ value):
                    runtime_error("PUT args:"),row(row),col(col),value(value){}

    const char* what(void){
        ostringstream oss;
        oss << "row =" << row << ", col= " << col << ", value = " << value;
        return (runtime_error::what() + oss.str()).c_str();
    }

private:
    int row, col;
    XYZ value;
};
//////////////////////////////////////////////////////
template <typename XYZ> class AtException : public runtime_error {
public:
    AtException(int row, int col):runtime_error("AT args:"),row(row),col(col){
    }

    const char* what(void){
        ostringstream oss;
        oss << "row =" << row << ", col= " << col;
        return (runtime_error::what() + oss.str()).c_str();
    }

private:
    int row, col;
    XYZ value;
};
```

Q3.

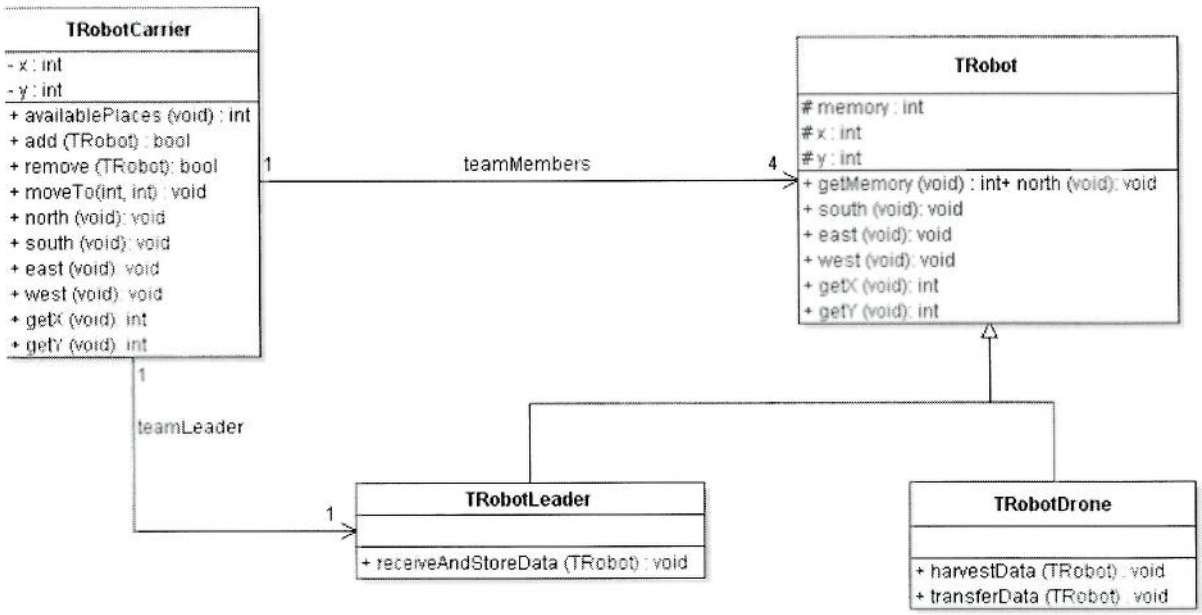
a) Based on a textual analysis of the problem description suggest names for four candidate classes. Briefly describe the role of each of your four suggestions.

[4]

`TRobot, TRobotDrone, TRobotLeader, TRobotCarrier.`
With a short description of the role of each class

b) Draw a UML class diagram that could be passed on to a programmer for implementation in code. Include as much detail as can be found in the problem description and no more. Name the classes and associations, include the association multiplicities and show appropriate visibilities for every class member. Include enough class members to clarify the role described in your answer to (a) but you do not need to include every class member.

[12]



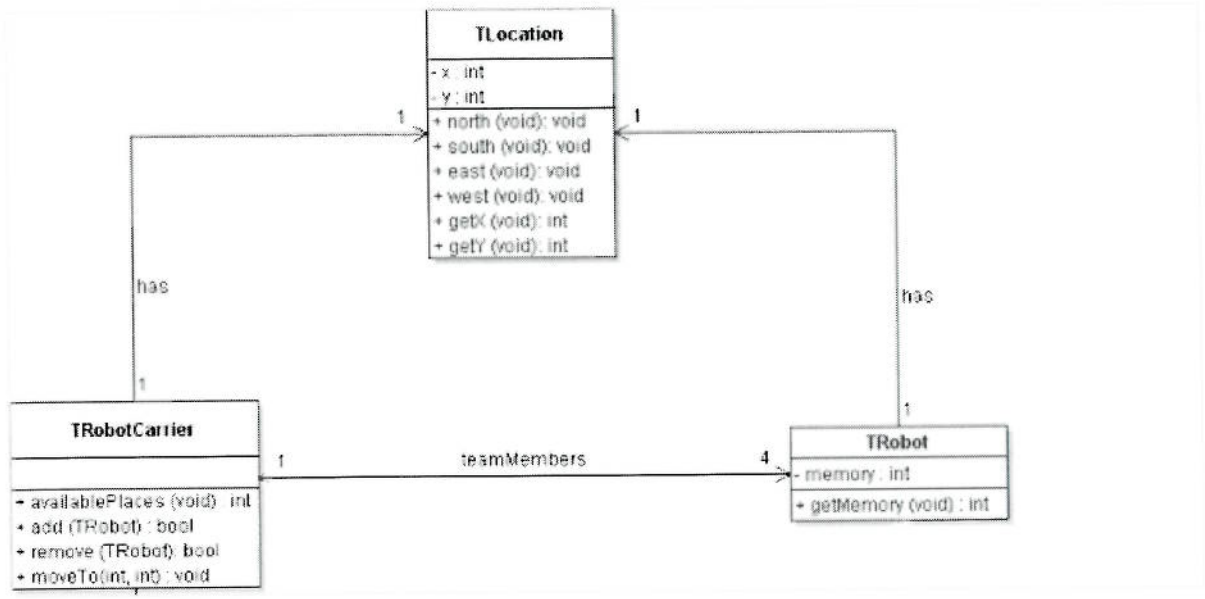
Note: The textual description is not clear enough about whether the associations are unidirectional or bidirectional, therefore either are acceptable answers.

Note: The UML class diagram semantics can not precisely specify that one of the 4 team members on the robot carrier must be a team leader. There is a language called the Object Constraint Language (OCL) that can be used to specify this level of detail using discrete mathematics. Since this was not taught in EE2-12 a variety of interpretations are acceptable answers. For example, a multiplicity of 3 instead of 4.

c) Two of the classes in your answer to (b) should have similar data members. These data members and the associated member functions can be factored out into a new class. Draw a *partial* class diagram that *only* includes the *new* class and the *two* classes identified in (b).

[10]

`TRobot` and `TRobotCarrier` share x, y position data members implying the need for a location utility class. The data members and associated member functions can be factored out into a new class that might be used in the construction of `TRobot` and `TRobotCarrier`. The use of the location utility class could be incorporated into the architecture using an association (as shown below) or using inheritance. Either is acceptable but as (d) reveals only one approach is valid.



d) Briefly argue the case either *for* or *against* the change of architecture as described in (c)

[4]

The answer to (d) depends on the architecture proposed in (c). Since either approach was acceptable, the conclusion and justification depend on the answer to (C).

- The use of a Generalisation: To gain the maximum benefit of inheritance and polymorphism the classes should be logically related. `TRobot` and `TRobotCarrier` do share position data members and are logically related. However, `TLocation` is not logically related to the aforementioned classes. So the utility class could be used for construction reasons but this is not a good motivation for inheritance. Thus inheritance is not the better approach.
- The use of an Association: It sounds right to say that each of the two robot classes each have a location i.e. the *has-A* test. Conversely it does not sound right that a robot/robot carrier is a location i.e. the *is-A* test. Therefore an association is the better approach as is shown in the class diagram for (c) above.